OPERATING SYSTEM
PRACTICAL - EXERCISE

1. Create a new process by invoking the appropriate system call. Get the process identifier of the
currently running process and its respective parent using system calls and display the same using a C program.

Program:

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>


int main() {
    pid_t pid;
    pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        return 1;
    }
    else if (pid == 0) {
        printf("[CHILD PROCESS]\n");
        printf("-> Child PID: %d\n", getpid());
        printf("-> Parent PID: %d\n\n", getppid());
    }
    else {
        printf("[PARENT PROCESS]\n");
        printf("-> Parent PID: %d\n", getpid());
        printf("-> Child's PID (returned by fork): %d\n", pid);
        printf("-> Parent's Parent (Grandparent) PID: %d\n\n", getppid());
    }

    return 0;
}
```

Output:

```
[PARENT PROCESS]
-> Parent PID: 107673
-> Child's PID (returned by fork):
107674
-> Parent's Parent (Grandparent)
PID: 107666

[CHILD PROCESS]
-> Child PID: 107674
-> Parent PID: 1
```

2. Identify the system calls to copy the content of one file to another and illustrate the same using a C program.

Program:

```c
int main()
{
    FILE *f1, *f2;
    char file1[100], file2[100];
    int c;

    printf("Enter source filename: ");
    scanf("%99s", file1);

    f1 = fopen(file1, "r");

    if (f1 == NULL)
    {
        printf("Cannot open source file");
        return 1;
    }

    printf("Enter destination filename: ");
    scanf("%99s", file2);

    f2 = fopen(file2, "w");

    if (f2 == NULL)
    {
        printf("Cannot open destination file");
        fclose(f1);
        return 1;
    }
```

```c
    while ((c = fgetc(f1)) != EOF)
    {
        fputc(c, f2);
    }

    printf("File copied successfully");

    fclose(f1);
    fclose(f2);

    return 0;
}
```

Output:

```
Enter source filename: source.txt
Enter destination filename: copy.txt
File copied successfully
```

3. Design a CPU scheduling program with C using First Come First Served technique with the following considerations.
a. All processes are activated at time 0.
b. Assume that no process waits on I/O devices.

Program:

```c
#include <stdio.h>

int main()
{
    int n, i;
    int bt[10], wt[10], tat[10];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst times:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &bt[i]);
    }

    wt[0] = 0;

    for (i = 1; i < n; i++)
    {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    for (i = 0; i < n; i++)
    {
        tat[i] = wt[i] + bt[i];
    }

    printf("\nProcess\tBurstTime\tWaitingTime\tTurnaroundTime\n");

    for (i = 0; i < n; i++)
    {
        printf("P%d\t\t\t%d\t\t\t%d\t\t\t%d\n",
               i + 1, bt[i], wt[i], tat[i]);
    }

    return 0;
}
```

Output:

```
Enter number of processes: 3
Enter burst times:
5 3 2

Process BurstTime   WaitingTime TurnaroundTime
P1          5           0           5
P2          3           5           8
P3          2           8           10
```

4. Construct a scheduling program with C that selects the waiting process with the smallest execution time to execute next.

Program:

```c
#include <stdio.h>

int main()
{
    int n, i, j, temp;
    int bt[10], wt[10], tat[10], p[10];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst times:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &bt[i]);
        p[i] = i + 1;
    }

    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (bt[i] > bt[j])
            {
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;

                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }

    wt[0] = 0;

    for (i = 1; i < n; i++)
        wt[i] = wt[i - 1] + bt[i - 1];

    for (i = 0; i < n; i++)
        tat[i] = wt[i] + bt[i];

    printf("\nProcess\tBurstTime\tWaitingTime\tTurnAroundTime\n");

    for (i = 0; i < n; i++)
        printf("P%d\t\t\t%d\t\t\t%d\t\t\t%d\n",
               p[i], bt[i], wt[i], tat[i]);

    return 0;
}
```

Output:

```
Enter number of processes: 3
Enter burst times:
5 2 3

Process BurstTime   WaitingTime TurnAroundTime
P2          2           0           2
P3          3           2           5
P1          5           5           10
```

5. Construct a scheduling program with C that selects the waiting process with the highest priority to execute next.

Program:

```c
#include <stdio.h>

int main() {
    int n, i, j, temp;
    int bt[10], p[10], wt[10], tat[10];

    printf("Enter the number of processes:");
    scanf("%d", &n);

    printf("\nEnter burst time and priority for each process:\n");

    for (i = 0; i < n; i++) {
        printf("P%d burst time:", i + 1);
        scanf("%d", &bt[i]);

        printf("\nP%d priority:", i + 1);
        scanf("%d", &p[i]);
        printf("\n");
    }

    // Sort according to priority
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (p[i] > p[j]) {
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;

                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }

```

```
38
39      for (i = 1; i < n; i++) {
40          wt[i] = wt[i - 1] + bt[i - 1];
41          tat[i] = wt[i] + bt[i];
42      }
43
44      float avgwt = 0, avgtat = 0;
45
46      printf("\nProcess Priority\tBurst Time\tWaiting Time\tTurnaround
    Time\n");
47
48      for (i = 0; i < n; i++) {
49          printf("P%d\t%d\t\t%d\t\t%d\t\t%d\n",
50                 i + 1, p[i], bt[i], wt[i], tat[i]);
51
52          avgwt += wt[i];
53          avgtat += tat[i];
54      }
55
56      printf("\nAverage Waiting Time = %.2f\n", avgwt / n);
57      printf("Average Turnaround Time = %.2f\n", avgtat / n);
58
59      return 0;
60  }
```

Output:

```
Enter the number of processes:1

Enter burst time and priority for each process:
P1 burst time:2

P1 priority:3


Process Priority    Burst Time  Waiting Time    Turnaround Time
P1  3       2       0       2

Average Waiting Time = 0.00
Average Turnaround Time = 2.00
```